

Review

Simulation-Based Development of Internet of Cyber-Things Using DEVS

Laurent Capocchi ¹, Bernard P. Zeigler ^{2,*} and Jean-Francois Santucci ¹

¹ SPE UMR CNRS 6134 Laboratoire, Campus Grimaldi, University of Corsica "Pasquale Paoli", 20250 Corte, France; capocchi@univ-corse.fr (L.C.); santucci@univ-corse.fr (J.-F.S.)

² RTSync Corp., 6909 W. Ray Road, Chandler, AZ 85226, USA

* Correspondence: zeigler@rtsync.com

Abstract

Simulation-based development is a structured approach that uses formal models to design and test system behavior before building the actual system. The Internet of Things (IoT) connects physical devices equipped with sensors and software to collect and exchange data. Cyber-Physical Systems (CPSs) integrate computing directly into physical processes to enable real-time control. This paper reviews the Discrete-Event System Specification (DEVS) formalism and explores how it can serve as a unified framework for designing, simulating, and implementing systems that combine IoT and CPS—referred to as the Internet of Cyber-Things (IoCT). Through case studies that include home automation, solar energy monitoring, conflict management, and swarm robotics, the paper reviews how DEVS enables construction of modular, scalable, and reusable models. The role of the System Entity Structure (SES) is also discussed, highlighting its contribution in organizing models and generating alternative system configurations. With this background as basis, the paper evaluates whether DEVS provides the necessary modeling power and continuity across stages to support the development of complex IoCT systems. The paper concludes that DEVS offers a robust and flexible foundation for developing IoCT systems, supporting both expressiveness and seamless transition from design to real-world deployment.

Keywords: IoCT; DEVS; SES; IoT; IoCT system design; modular design; IoCT simulation frameworks



Academic Editor: Paolo Bellavista

Received: 14 May 2025

Revised: 23 June 2025

Accepted: 27 June 2025

Published: 30 June 2025

Citation: Capocchi, L.; Zeigler, B.P.; Santucci, J.-F. Simulation-Based Development of Internet of Cyber-Things Using DEVS. *Computers* **2025**, *14*, 258. <https://doi.org/10.3390/computers14070258>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

The Internet of Cyber-Things (IoCT) [1] represents a convergence of the Internet of Things (IoT) and Cyber-Physical Systems (CPSs), emphasizing not only the connectivity of smart devices but also their integration with cyber-enabled control over physical processes. This paradigm envisions a network where digital and physical realms interact seamlessly, enabling real-time management and optimization of physical systems [1]. In such environments, information flows continuously between sensors and actuators under the supervision of computational intelligence [1,2].

Simulation-based development provides a powerful methodology for designing and analyzing complex systems by enabling the creation of virtual models that replicate system behavior under various conditions. This approach allows for rapid iteration, early validation, and performance optimization. It also facilitates the exploration of interactions between software components and physical entities, ensuring that real-time constraints are

met. By simulating diverse scenarios—including failures, delays, and environmental variations—developers can identify and resolve design flaws before deployment, significantly reducing the cost and risk associated with physical prototyping.

IoCT systems combine the loosely coupled nature of IoT with the stringent real-time and control requirements of CPS. As such, they demand modeling approaches that can accurately represent discrete-event interactions, support real-time decision-making, and adapt to evolving system configurations. Simulation plays a critical role in addressing these challenges by modeling communication flows, control logic, and system dynamics as discrete events.

To effectively support the development of IoCT systems, a modeling language must satisfy two essential properties:

- *Model expressiveness*, which refers to the ability to represent complex system behaviors, including hybrid dynamics that combine discrete and continuous processes. It also includes support for modularity and incremental model refinement.
- *Model continuity*, which ensures that the same model can be used consistently across the design, simulation, and implementation stages, with minimal modification. This is particularly important for preserving timing accuracy and behavioral fidelity in real-world deployments.

The Discrete-Event System Specification (DEVS) formalism [3] offers a rigorous framework for modeling systems as discrete-event processes. Its modular and hierarchical structure allows for the composition of atomic models into larger coupled systems, making it well suited for both asynchronous IoT interactions and real-time CPS control. DEVS supports dynamic structural changes, enabling the modeling of adaptive and autonomous behaviors [4–6]. Atomic models define the behavior of individual components, while coupling mechanisms allow for scalable system integration.

This paper demonstrates that DEVS satisfies the requirements of model expressiveness and continuity, establishing it as a strong candidate for IoCT system design. Through a series of illustrative case studies, the capabilities of DEVS in enabling scalable, flexible, and reusable modeling are evidenced. Additionally, the System Entity Structure (SES) plays a significant role in enhancing model organization and facilitating the generation of alternative system configurations.

Figure 1 illustrates the stages and requirements of an IoCT design language. On the left, an IoCT system is shown interacting with its environment via sensors and actuators. On the right, the three core development stages—design, simulation, and implementation—are depicted. While current design tools [7] often lack support for seamless model continuity, DEVS addresses this gap by enabling hierarchical model construction, reuse, and simulation across both discrete-event and continuous domains [8]. Furthermore, DEVS models can be transitioned from abstract to real-time execution with minimal changes [9,10], ensuring consistency throughout the development lifecycle. The case studies presented in this paper help demonstrate the effectiveness of the DEVS formalism in expressing the system at the design level, as well as its continuity and ability to be used almost as-is in the final implementation phase.

The remainder of this paper is organized as follows. Section 2 provides a review of the DEVS formalism and the SES, which serve as foundational frameworks for the modeling and simulation (M&S) of complex systems. Section 3 presents requirements for IoCT design and development and summarizes how DEVS serves as a language to help users address these requirements. Section 4 presents a series of illustrative examples demonstrating the application of DEVS in designing IoCT systems. These examples include home automation, the CAIDE architecture, the Actuation Conflict Management (ACM) framework, and swarm-based systems. Section 5 offers a discussion on how the presented examples

highlight the expressiveness of DEVS for capturing dynamic and complex behaviors and its continuity in seamlessly transitioning simulation models to real-world implementations. It also compares these features with those of other widely used formalisms. Finally, Section 6 concludes the paper by summarizing the key findings and discussing limitations of DEVS-based IoCT system design and discusses how model-driven engineering can help address such limitations. Finally, future directions are outlined for further leveraging DEVS in the design of IoCT systems.

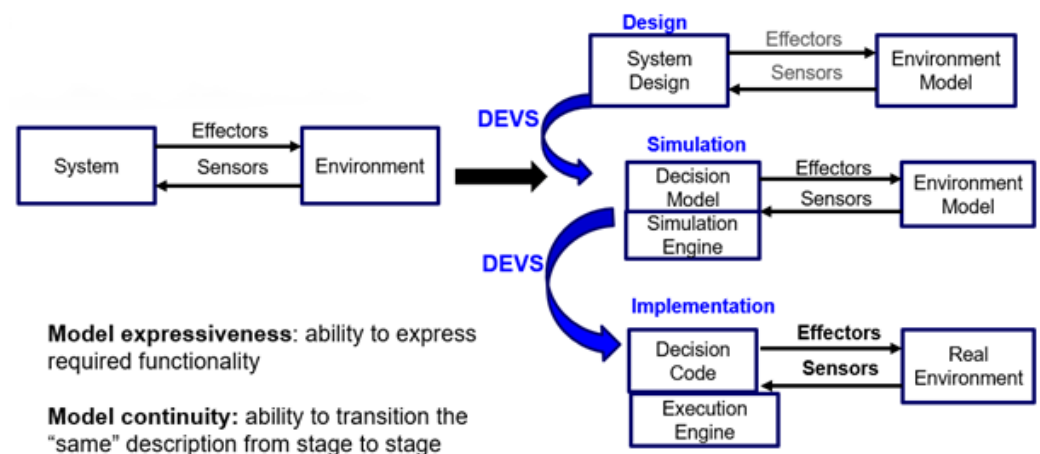


Figure 1. Stages and the requirements for an IoCT design language. The DEVS formalism provides the model expressiveness needed for IoCT functional design and supports the model continuity needed to transition from stage to stage. Left, an IoCT system is shown interacting with its environment via sensors and actuators. Right, the three core development stages.

2. Background and Related Work

The IoCT combines the connectivity of IoT with the control and real-time requirements of CPS. Modeling such systems requires both expressiveness and continuity across the development lifecycle. While the DEVS formalism provides a strong foundation for this purpose, it is important to situate it within the broader landscape of modeling approaches for IoT and IoCT systems.

DEVS offers modularity, hierarchy, and formal semantics for discrete-event modeling. It supports both atomic and coupled models, and its dynamic structure extension (DS-DEVS) enables runtime reconfiguration. The System Entity Structure (SES) complements DEVS by organizing model variants and supporting automated configuration. A detailed overview of DEVS and SES is provided in Appendix A.

Several alternative modeling approaches have been proposed for IoT systems. Finite-state machines (FSMs) are widely used for control logic but lack support for concurrency and hierarchical composition. Petri nets are effective for modeling concurrency and synchronization but become complex in large systems and lack native support for dynamic structure. SysML and UML-based approaches are popular in industry for system-level modeling, but they often lack integrated simulation capabilities and real-time execution support.

Recent frameworks such as Node-RED and ThingML provide visual or model-driven development environments for IoT applications. However, they typically focus on implementation and orchestration rather than formal modeling and simulation. In contrast, DEVS enables early validation through simulation and supports seamless transition to deployment, as demonstrated in the case studies presented in this paper.

In summary, while many modeling approaches address specific aspects of IoT system design, DEVS stands out for its ability to unify design, simulation, and execution within a

single formalism, making it particularly suitable for the complex and evolving nature of IoCT systems.

3. A Unified DEVS-Based Methodology for Expressive and Continuous IoCT Design

The DEVS formalism offers several key features—modularity, event-driven behavior, concurrency, adaptability, scalability, and model continuity—that make it particularly well suited for designing and developing IoCT systems. These systems are inherently complex and heterogeneous, involving many types of devices and components that must work together reliably.

To cope with this complexity, a modeling approach must address several essential requirements (see Figure 2):

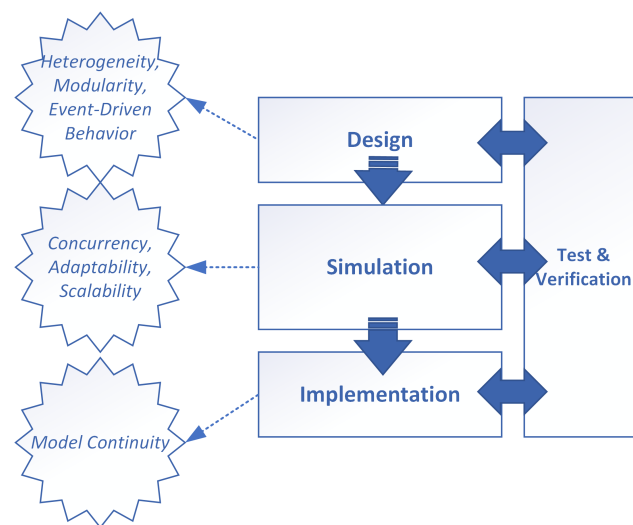


Figure 2. DEVS-based development methodology for IoCT systems. The process is structured into three main stages: *Design*, *Simulation*, and *Implementation*. Each stage is associated with key DEVS properties—modularity, event-driven behavior, concurrency, adaptability, scalability, and model continuity—highlighting DEVS’s ability to support a seamless transition from conceptual modeling to real-world deployment.

- **Heterogeneity:** IoCT systems include a wide variety of devices, sensors, and actuators, each using different communication protocols, data formats, and processing capabilities. A suitable modeling formalism must be flexible enough to represent this diversity and ensure smooth interaction between all components.
- **Real-Time and Event-Driven Behavior:** Many IoCT applications require immediate responses to sensor data. For instance, a smart thermostat must react quickly to temperature changes, and a traffic control system must process sensor inputs in real time to manage vehicle flow efficiently.
- **Concurrency and Synchronization:** IoCT systems often involve multiple devices operating at the same time. These devices must coordinate their actions to avoid conflicts and ensure that the system behaves as expected. A modeling approach must support concurrent execution and synchronization mechanisms.
- **Adaptability:** IoCT environments are dynamic—devices may fail, new ones may be added, or network conditions may change. The modeling formalism must support flexible structures that can adapt to these changes without requiring a complete redesign.
- **Scalability:** As IoCT systems grow, the modeling approach must allow for the addition of new components without compromising performance or stability. This is essential for systems that evolve over time or are deployed at large scale.

- **Model-to-Execution Continuity:** A model should remain consistent from the design phase through simulation and into real-world deployment. This ensures that the behavior tested during simulation is preserved when the system is implemented.

DEVS addresses these requirements through its modular and hierarchical structure, which allows complex systems to be built from smaller, reusable components. Its event-driven nature makes it ideal for modeling time-sensitive behaviors, such as those triggered by sensor inputs. DEVS also supports concurrency by allowing multiple components to operate in parallel and synchronize their actions. Its support for dynamic structure enables systems to adapt to changes in their environment, such as the appearance or disappearance of devices. Scalability is achieved through incremental model construction, and model continuity ensures that the same model can be used throughout the development lifecycle—from design to simulation to execution—without major modifications.

4. Case Studies and Applications

This section is dedicated to presenting several examples highlighting the enhanced expressiveness and continuity offered by DEVS in the field of IoCT systems.

4.1. Home Automation

The suitability of DEVS for modeling and deploying IoCT applications has been demonstrated in various studies, particularly in the domain of home automation [11]. Faizel and Wainer [8] proposed a DEVS-based architecture that captures the interaction between multiple sensors and actuators in a smart environment. Their approach involved defining atomic DEVS models for key operations such as sensor polling, data transmission, and data sharing. These atomic models were hierarchically composed into a coupled model capable of executing on real-world hardware. To handle communication across devices, they introduced a network of interconnected nodes modeled as a higher-level coupled DEVS system. The simulation results confirmed the architecture's ability to aggregate sensor data and coordinate inter-device communication to reach coherent system-level decisions. Quantitative validation of DEVS-based IoT architectures is given in a home automation context. Their experiments demonstrated that the DEVS-based system achieved a 22.86% higher precision, 24.05% higher recall, and 23.45% higher F1-score compared to a traditional 2D classification method for sensor fusion. These improvements were observed in scenarios involving multiple sensors and actuators, where the DEVS formalism enabled more accurate and timely decision-making. These results support the claim that DEVS not only facilitates modular and hierarchical modeling but also enhances the operational performance of IoT systems in real-world deployments.

Crucially, their methodology extended beyond simulation: the same DEVS models used in the design phase were deployed on physical microcontrollers, demonstrating the principle of model continuity. Using Cadmium Version 2, a C++-based DEVS simulation engine, the models were ported to embedded firmware with minimal changes. Aside from substituting the message broker model with a commercial implementation, the transition from simulation to deployment required only minor code modifications. This reuse of models across the design lifecycle emphasizes the practical utility of DEVS for seamless simulation-to-implementation workflows.

Traditionally, the development of IoCT systems involves a sequential process: defining component behavior libraries, establishing interconnections, and executing the assembled system. However, when timing or coordination issues emerge late in the process, revisiting and modifying the initial models becomes costly and time-intensive. Sehili et al. [12] proposed a more iterative and simulation-driven workflow, where behavioral and control models are evaluated early in the development cycle using DEVS. Focusing on a lighting

control scenario, they demonstrated how DEVS can encode both system behavior and execution logic, functioning analogously to a finite-state machine. This flexibility allows developers to define, compare, and validate alternative execution strategies by adjusting model configurations prior to implementation. By identifying and resolving conflicts during the simulation phase, designers can proceed to deployment with higher confidence and reduced debugging effort. The benefits of this strategy for resolving access conflicts and improving system coordination are further elaborated in Section 4.3.

Two key features illustrated by these applications underscore the strengths of DEVS in IoCT system development. First, model expressiveness is enabled by the formalism's support for modular, hierarchical system composition and its ability to interface with dynamically changing environments. Second, model continuity is facilitated through the use of portable DEVS execution engines, such as Cadmium V2 (C++) and Python, allowing the same models to be reused for both simulation and embedded deployment. Together, these properties demonstrate the value of DEVS in bridging the gap between conceptual modeling and real-world implementation in smart home and broader IoCT scenarios.

4.2. Solar Monitoring

Moving from simulation to real-world deployment shows how useful DEVS is for connecting system design with actual implementation, especially in solar monitoring and other IoCT applications.

The Cloud-Based Analysis and Integration for Data Efficiency (CAIDE) framework, developed using the DEVS formalism, demonstrates the capacity of DEVS to support complex IoCT applications in the renewable energy domain, particularly in solar irradiance sensor farms [13]. Integrating DEVS with Model-Based Systems Engineering (MBSE) [14] and a cloud-enabled IoT infrastructure, CAIDE enables the deployment, monitoring, and continuous optimization of solar plants in dynamic and distributed environments. A defining feature of the framework is its ability to manage multiple farms concurrently while dynamically adapting and retraining predictive models in real time, thereby maintaining the accuracy and responsiveness of solar irradiance forecasts under evolving environmental conditions.

As illustrated in Figure 3, the system architecture is structured as a hierarchical DEVS coupled model, comprising two principal subsystems: local farms and a cloud-based training environment supported by big data services. Each farm encapsulates atomic DEVS models responsible for data acquisition from solar sensors, preprocessing, and coordination via a central Fog server. This modular architecture enables execution across sequential, parallel, and distributed computing platforms, offering a high degree of scalability. The implementation was validated through a realistic scenario involving multiple sensor farms interfaced with a centralized management system, showcasing the applicability of DEVS in operational deployments of solar monitoring infrastructures and its broader implications for intelligent energy systems.

This application further exemplifies the dual strengths of DEVS in terms of model expressiveness and continuity. On the one hand, DEVS provides a formal basis for expressing the layered architecture of CAIDE—encompassing sensor, Fog, and cloud layers—as well as the temporal behavior of sensor interactions and the modular AI/ML components required for adaptive analysis. On the other hand, model continuity is achieved through the reuse of the same DEVS models across development and deployment phases. Initial simulation and design were conducted using a Python-based DEVS engine, while real-time execution was realized with minimal modifications, preserving the fidelity of the original models. This seamless transition from simulation to operational deployment reinforces the

practical value of DEVS in bridging system design and implementation in the context of solar monitoring and broader IoCT applications.

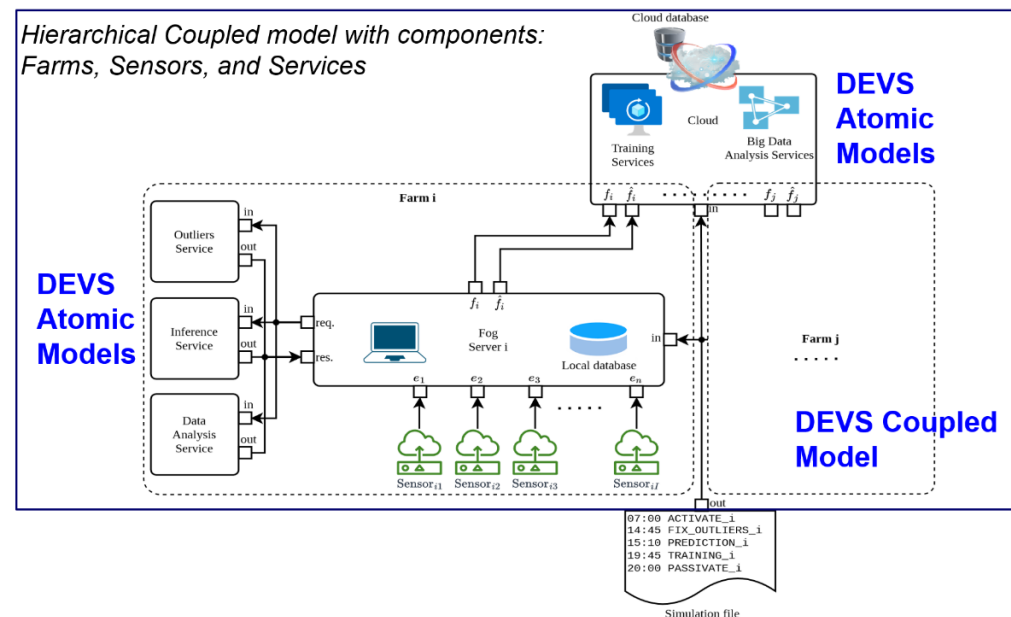


Figure 3. IoCT Solar System Architecture designed and implemented using DEVS, highlighting support for expressiveness and continuity.

4.3. Actuation Conflict Management

In managing IoT conflicts, rigorous validation is essential to detect issues by analyzing both the events generated by smart applications and the resulting actions on shared actuators [15–17]. As these applications increasingly control common IoT devices—especially actuators that produce tangible physical effects—the risk of actuation conflicts grows. These conflicts manifest either directly, when multiple applications vie for the same actuator, or indirectly, when applications influence shared physical properties.

To mitigate these challenges, actuation conflicts must be addressed during the design phase, rather than relying on end-users to resolve them. The goal is to implement Actuation Conflict Management (ACM) mechanisms that proactively identify and resolve both direct and indirect conflicts early in the system’s lifecycle (see Figure 4). Validation plays a critical role in this process by simulating IoT application events and actuator responses, ensuring that the ACM specifications remain robust and effective in real-world scenarios.

This work builds upon the M&S approach outlined in recent research [18], proposing a method that integrates M&S into the design of IoT systems. The methodology leverages the DEVS formalism within the DEVSimPy multi-platform framework [19,20], facilitating a robust and flexible system design. DEVSimPy is a cross-platform wxPython-based GUI for modeling and simulating DEVS-based systems. It allows users to build DEVS models by interconnecting atomic and coupled models instantiated from libraries [21].

Figure 4 illustrates a custom ACM design process structured into three main levels:

- **Logical ACM Design:** At this stage, logical properties of custom ACMs—such as completeness, safety, and liveness—are formally verified using techniques like model checking [22,23]. ACMs are defined by designers as finite-state machines (FSMs), forming the foundation for validation.
- **Model Validation:** This level assesses the impact of conflict resolution on the environment using DEVS simulation. The ACM DEVS model incorporates timing properties (e.g., event delays and state durations) to accurately capture system behavior.

- **Model Deployment:** Temporal properties of the custom DEVS ACMs are verified using various asynchronous execution strategies. DEVS enables experimentation with different middleware/edge platforms (e.g., Node-RED [24] and ThingML [25]) by generating platform-specific simulation kernels.

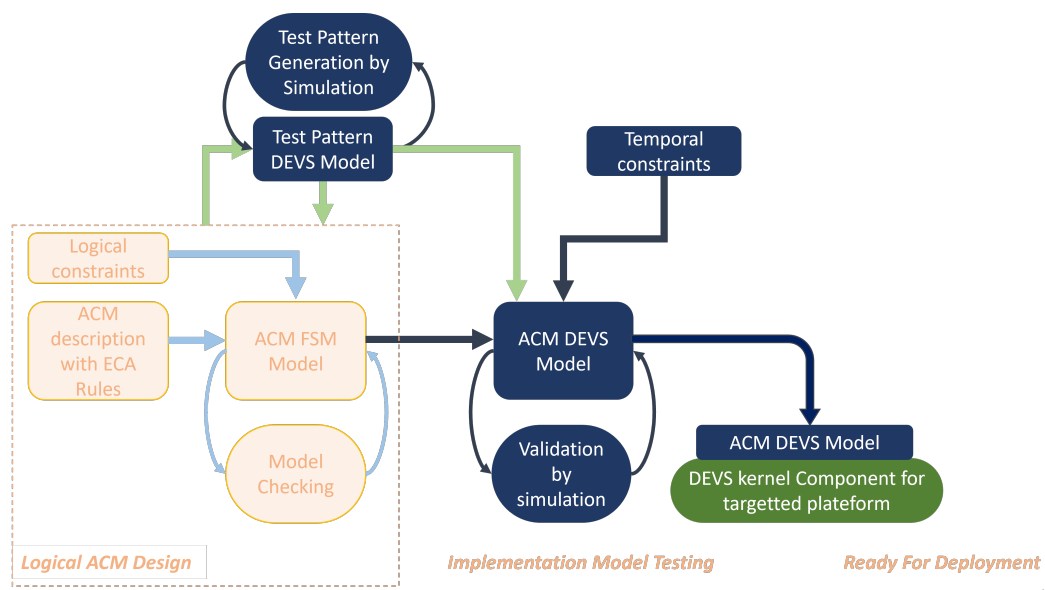


Figure 4. DEVS M&S inside the custom ACM design process with its three levels: logical ACM design, model validation, and model deployment.

This simulation-based ACM approach intercepts all interactions between application flows and actuators to detect both direct and indirect conflicts and validate resolution strategies through a dedicated ACM component (Figure 5c).

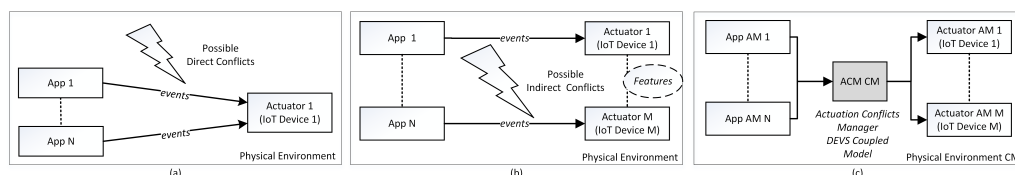


Figure 5. IoCT systems face two conflict types: (a) direct conflicts from multiple flows targeting a shared actuator; (b) indirect conflicts via shared environmental effects; (c) resolution using a DEVS-based ACM model.

DEVS models were used early in the ACM design process to validate conflict detection mechanisms through simulation. The ACM coupled model includes two atomic DEVS models: a *Synchronizer*, which processes application events, and a *Logical Behavior* model that encodes conflict resolution logic (Figure 6).

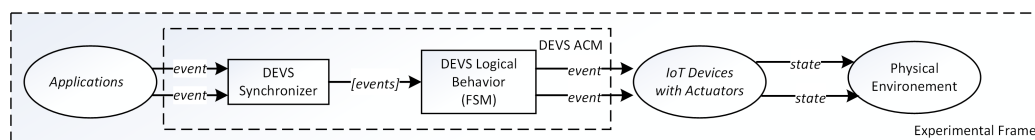


Figure 6. The ACM model with Synchronizer and Logical Behavior components embedded in a Physical Environment comprising application flows and actuators.

Designers use a Logical Behavior model, typically an FSM, to express rules and transitions. However, synchronization and timing issues are not covered by traditional FSM validation. The Synchronizer DEVS model plays a crucial role by managing inputs

from IoT flows based on predefined strategies (e.g., waiting for all inputs, triggering by timer, or sending outputs immediately).

Figure 7 depicts the ACM DEVS model applied in a Smart Home scenario using DEVSimPy. Here, 216 application flows control 37 actuators (windows, lights, ACs, etc.). Conflicts such as simultaneous control of the same device (e.g., a TV) are detected and resolved. A total of 3124 direct and 673 indirect conflicts were identified and successfully handled.

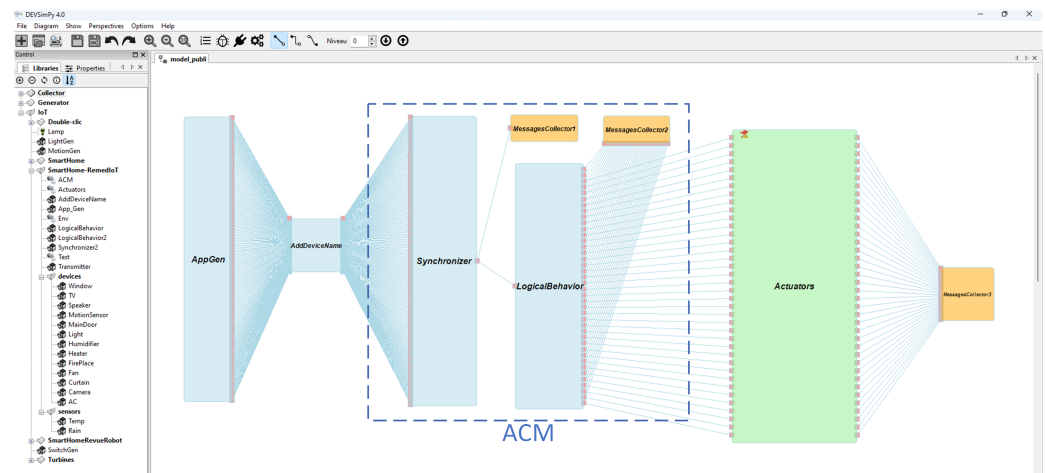


Figure 7. DEVSimPy model of a Smart Home with ACM integrated. The Synchronizer and Logical-Behavior atomic models are part of the ACM coupled model.

DEVS model continuity enables the ACM simulation engine to be mapped to middle-ware like Node-RED and ThingML, supporting orchestration and deployment across edge and cloud infrastructure. The DEVS simulation kernel’s portability facilitates deployment on various platforms [9,12].

4.4. Smart Parking

Sensor access conflicts also arise prominently in smart parking systems, which aim to optimize the utilization of parking spaces by integrating driver behavior with real-time sensor data. A central challenge in this domain is the development of robust models capable of managing cumulative parking conflicts—situations where multiple drivers simultaneously compete for available spaces in a dynamic environment influenced by both user decisions and sensor inputs. Modern sensor networks enable real-time monitoring of parking space availability, providing critical data for such models. For instance, Domini et al. [26] propose a DEVS-based modeling and simulation framework that devises conflict management strategies grounded on estimated travel times to targeted parking locations within a defined area. DEVS is particularly suited to this application due to two key advantages: (i) the explicit representation of elapsed time as an essential state variable, governed by the time-advance function, which affords precise temporal control in both simulation and real-time contexts; and (ii) its hierarchical and modular architecture, facilitating the construction and reuse of conflict models tailored to individual drivers across diverse scenarios.

Figure 8 illustrates the DEVSimPy simulation model employed in this smart parking system, encompassing ten drivers and ten parking spaces. The model structure begins with Space atomic models—each representing an individual sensor—which are aggregated into a Zone coupled model. Sensor data are collected and processed by the UserLink atomic model and relayed to driver models, which filter and evaluate available parking spaces based on predefined criteria. This filtered information informs the Travel model, which

applies decision policies to select an optimal parking space. The selection is then passed to the AccessConflictManagement model, responsible for resolving contention among drivers through various conflict resolution algorithms.

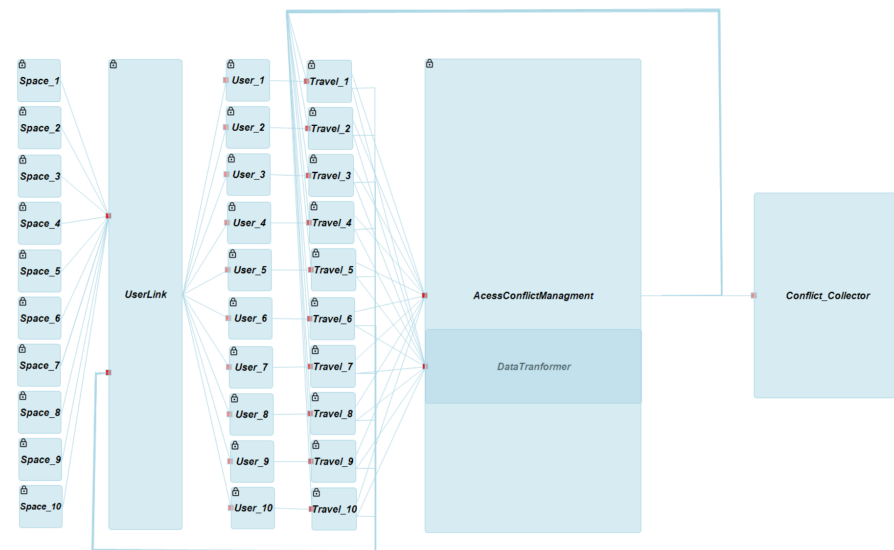


Figure 8. The DEVSImPy simulation model integrates all the DEVS atomic models along with their interconnections. The simulation considers 10 drivers and 10 parking spaces. A Conflict_Collector model is employed to gather simulation outputs, which are subsequently used for result analysis.

The DEVS-based smart parking system has been deployed in the city of Bastia (Corsica, France), where over 450 sensors were installed to monitor parking availability. This includes 340 standard sensors and 110 sensors for limited-time parking zones. Initially, due to the lack of historical data, a DEVS simulation model was used to generate synthetic datasets based on Poisson and constraint-based distributions. These datasets were used to train a neural network (NN) model for predicting parking slot occupancy durations.

Once real data became available—over one million parking events collected over a year—the same NN model was applied to the real dataset. A comparative analysis using dendrograms and heatmaps showed strong structural similarity between simulated and real data distributions. The NN model, initially trained on simulated data, achieved 96.6% accuracy (Table 1). When applied to real data, the accuracy decreased slightly, but the model remained effective. The simulation phase was instrumental in identifying key behavioral constraints (e.g., billing time, user profiles, and daily patterns) that were later integrated into the real-time prediction system.

Table 1. Evolution of neural network accuracy across simulation phases.

Simulation Phase	Accuracy (%)
Basic Poisson Simulation	85.2
Constraint-Based Simulation	90.4
Daily Behavior Consideration	93.1
Billing Time Integration	95.7
Final Accuracy on Real Data	96.6

This deployment illustrates the practical benefits of using DEVS not only for system design and simulation but also for preparing machine learning models for real-world smart city applications.

DEVS played a central role in addressing real engineering challenges by enabling a seamless transition from simulation to deployment in the smart parking system implemented in Bastia, France. Initially, no historical data were available, which is a common obstacle in real-world IoT projects. DEVS was used to simulate realistic parking behaviors using constraint-based Poisson models, allowing us to generate synthetic datasets for training a neural network. This simulation phase was not just theoretical—it directly informed the architecture and configuration of the machine learning model used in production.

Once real data became available (over one million parking events), the same DEVS-based model was reused without structural modification, demonstrating model continuity. The simulation also helped identify key behavioral constraints (e.g., billing time, user profiles, and daily patterns) that significantly improved prediction accuracy. The final model achieved 96.6% accuracy on simulated data and remained effective on real data. This would not have been possible without the flexibility, modularity, and real-time execution capabilities of DEVS.

In short, DEVS was not only a modeling tool—it was a critical enabler of system design, data generation, model training, and operational deployment in a real smart city context.

The DEVS model's continuity is demonstrated by its seamless integration with an API providing real-time sensor data from operational parking facilities. By incorporating user authentication and switching the DEVS kernel to real-time execution mode, the framework was deployed within a mobile application for the city of Bastia (Corsica, France), as depicted in Figure 9. This deployment exemplifies how DEVS supports the transition from simulation to practical IoCT applications, enabling drivers to locate and occupy available parking spaces based on live data streams.

In this case study, both the model expressiveness and model continuity of the DEVS paradigm are further illustrated in a practical context. The ability to represent concurrent streams of temporal events enables simulation-based validation of actuator coordination mechanisms before deployment, ensuring smooth integration within the physical environment. In parallel, model continuity is demonstrated through the reuse of the exact same DEVS model—from the initial design phase to the mobile application deployed in the field—without requiring structural modification. This reuse is facilitated by the DEVS simulation engine's compatibility with middleware used in IoCT infrastructures, allowing for straightforward integration. Moreover, its adaptability across heterogeneous hardware platforms with different timing characteristics reinforces its portability. Most importantly, DEVS supports a seamless transition from simulation to real-world implementation, enabling validated models to be deployed in operational environments with minimal effort.

4.5. Swarm Systems

In [27], Zhang et al. emphasize the expressive power of the DEVS formalism, particularly its dynamic structure capability, in the M&S of *unmanned swarm systems* (USSs). These systems find broad applicability across diverse domains including military operations, precision agriculture, and aerospace. The authors highlight the limitations of traditional modeling approaches in capturing the complex and adaptive dynamics of USS and demonstrate how DEVS can effectively be utilized to design, simulate, and implement such IoCT systems. DEVS facilitates a dual-perspective description of unmanned platforms: atomic DEVS models represent microscopic behaviors of individual components, while the coupled DEVS framework captures the collaborative and hierarchical structure among the unmanned platforms.

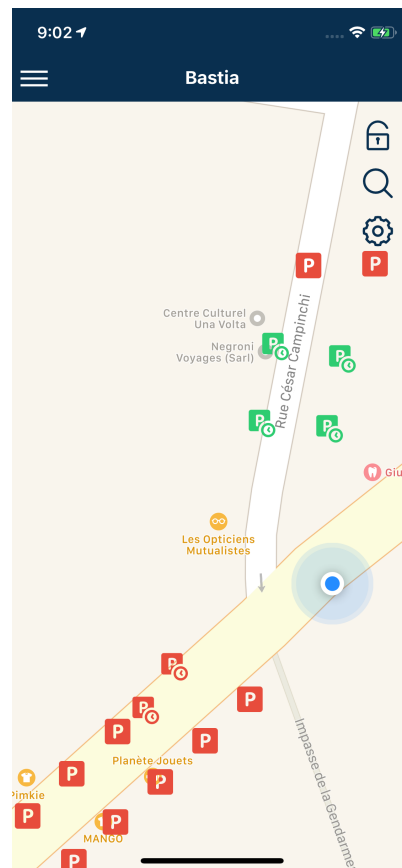


Figure 9. Mobile app that embeds the smart parking DEVS simulation model for the Bastia city (Corsica–France), which is equipped with more than 400 presence sensors on the roads.

The dynamic structure capability of the DEVS formalism supports swarm growth and self-optimization for surveillance missions by enabling the system’s structure to evolve throughout simulation. This capability allows for the dynamic addition or removal of sub-models and ports, as well as the modification of port connections, thus empowering the USS to autonomously restructure in response to environmental changes according to predefined structural change rules and system goals. Underpinning these adaptive behaviors is a DEVS-based synchronization mechanism that coordinates actions and structural transformations across the swarm.

Figure 10 shows how the USS and its environment are structured using a hierarchical and modular approach. At the top, the SES represents the abstract organization of the system, while the bottom illustrates the corresponding hierarchical DEVS model. The swarm is composed of a central Controller and several Unmanned Autonomous Vehicles (UAVs), each modeled as a coupled DEVS component that includes atomic models for movement and terrain sensing. The system demonstrates adaptive reconnaissance behavior: UAVs can be deployed to fill surveillance gaps, withdraw if critically damaged, move to avoid interference, or resolve overlapping coverage areas based on their damage level. These behaviors are enabled by the dynamic structure capabilities of DEVS, which allow the system’s configuration to evolve during simulation or operation. In addition, DEVS’s temporal semantics support the synchronization needed for coordinating UAV actions, especially when task durations vary due to environmental factors or different rule applications. Synchronization is achieved by having the Controller wait for all UAVs to report task completion before proceeding.

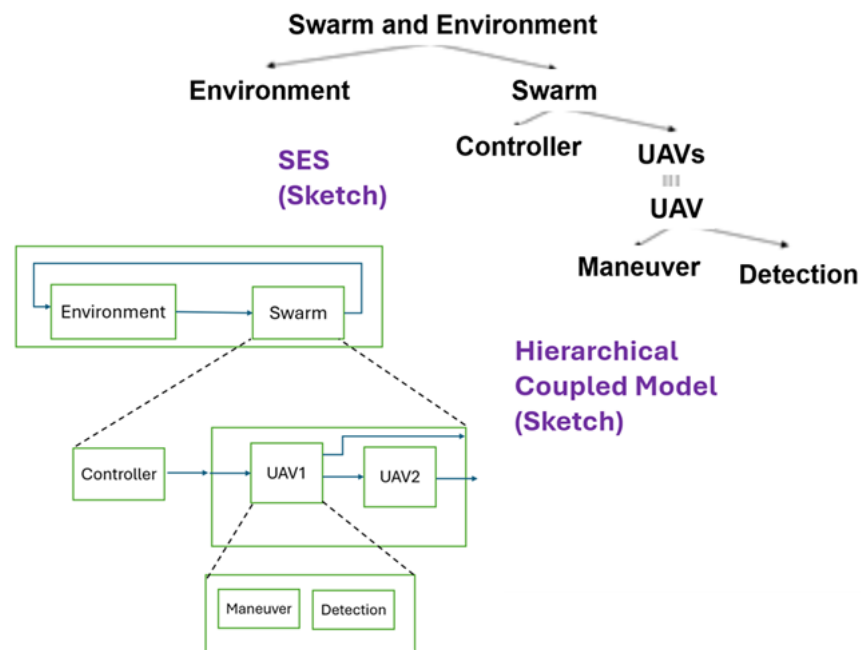


Figure 10. Hierarchical, modular structure of an unmanned swarm system and its environment represented by the SES (top) and the corresponding hierarchical coupled DEVS model (bottom). Figure copied and modified with permission.

Figure 11 employs the SES formalism to illustrate decentralized synchronization variants, where the central Controller is removed and UAVs autonomously implement synchronization protocols. While DEVS effectively expresses synchronization behavior in both centralized and decentralized architectures, SES provides a clear representation of alternative system decompositions, highlighting distinct sub-trees for each synchronization approach.

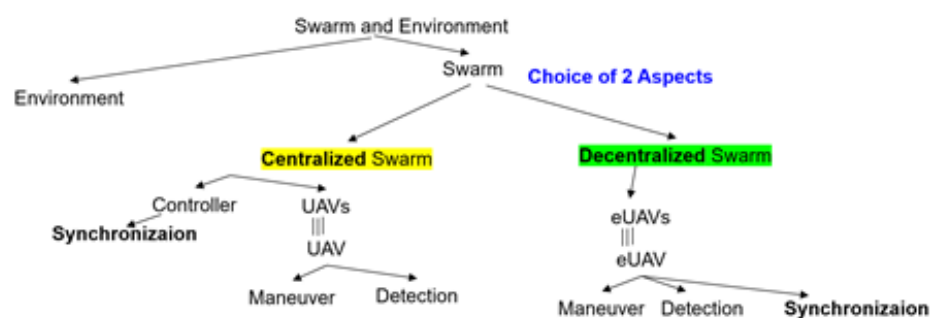


Figure 11. SES illustrating centralized and decentralized synchronization alternatives in USS. The SES details are further elaborated in Figure 12.

Figure 12 presents a fragment of the SES textual description. The *topLevel* aspect decomposes the overall *SwarmNEnvironment* model into *UAVSwarm* and *Environment* components. Coupling relations define connections between component output and input ports. The yellow and green highlights correspond to the two alternative decompositions of *UAVSwarm*, as shown in Figure 11, specifying external input and output couplings that capture distinct architectural designs.

```

From the topLevel perspective, SwarmNEnvironment is made of UAVSwarm and Environment!
From the topLevel perspective, UAVSwarm sends outCommand to Environment as inCommand!
From the topLevel perspective, Environment sends outCommand to UAVSwarm as inCommand!
From the topLevel perspective, UAVSwarm sends outStatusInformation to Environment as
inStatusInformation!
From the topLevel perspective, Environment sends outStatusInfoCorrupted to UAVSwarm as
inStatusInfoCorrupted!

From the Centralized perspective, UAVSwarm is made of Controller, First_UAV, Second_UAV, and
Third_UAV!
From the Centralized perspective, UAVSwarm sends inCommand to all UAV as inCommand!
From the Centralized perspective, Controller sends outCommand to UAVSwarm as outCommand!
From the Centralized perspective, UAVSwarm sends inStatusInfoCorrupted to Controller as
inStatusInfoCorrupted!
From the Centralized perspective, all UAV sends outStatusInformation to UAVSwarm as
outStatusInformation!

From the Decentralized perspective, UAVSwarm is made of Second UAV, First UAV, and Third UAV!
From the Decentralized perspective, all UAV sends outStatusInformation to all UAV as inStatusInformation!
From the Decentralized perspective, all UAV sends inCommand to all UAV as inCommand!

```

Figure 12. Excerpt of the SES textual description using constrained natural language, specifying hierarchical coupled models constructed from two alternative aspects—centralized (yellow) and decentralized (green) control.

The strengths of DEVS for modeling unmanned swarm systems are evident in several key aspects. Its model expressiveness is demonstrated through the capability of dynamic structure adaptation at runtime, which enables changes in the model topology to reflect system reconfiguration seamlessly. Additionally, the hierarchical and modular nature of DEVS supports a clear decomposition into atomic and coupled models, effectively capturing both behavioral and structural system characteristics. The intrinsic temporal semantics of DEVS further facilitate synchronization mechanisms that are critical for coordinating heterogeneous task execution among UAVs. Regarding model continuity, the integration of the SES formalism with DEVS allows for alternative architectural decompositions, such as centralized and decentralized control schemes, without altering the core model semantics. This abstraction at a high level also promotes direct reuse of simulation models for real-time implementation and comprehensive system validation.

4.6. Cyber-IoT Integration in IoCT

While IoT design primarily addresses the communication and control of sensors and actuators, Cyber-Physical System (CPS) design focuses on the intricate interactions between digital and physical components [28]. As applications grow increasingly complex and demanding, these two paradigms are converging towards a unified methodology known as the IoCT, which synergistically integrates their respective strengths [29].

Figure 13 illustrates an architecture that leverages the DEVS formalism to define modular building blocks encompassing both computational and control-theoretic functionalities fundamental to intelligent CPS design [30]. A practical realization of this approach, described in [31], exploits the continuity properties of DEVS models to efficiently develop embedded controllers for robotic systems. This same property underpins the design, simulation, and implementation of analogous constructs in Unmanned System and Subsystem (USS) domains.

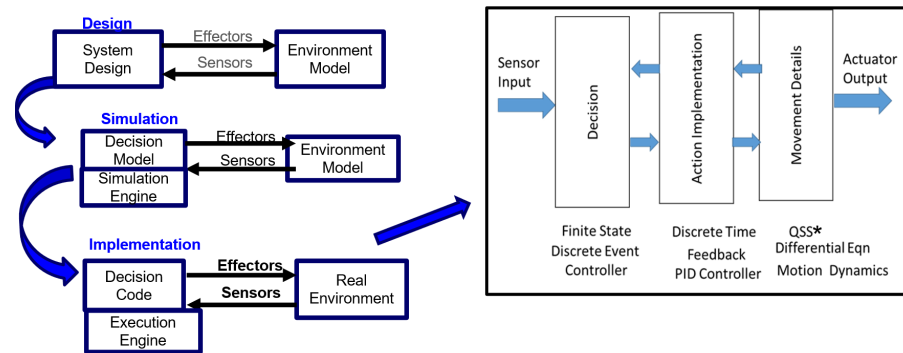


Figure 13. Architecture that leverages the DEVS formalism to define modular building blocks encompassing both computational and control-theoretic functionalities fundamental to intelligent CPS design [30].

Within this context, two core DEVS features demonstrate particular relevance to model expressiveness and continuity. Regarding expressiveness, atomic DEVS functions encapsulate fine-grained behaviors such as message passing, timing, and decision-making processes. The framework’s atomic and coupled models inherently support synchronization, while its hierarchical and modular structure facilitates the representation of collaborative interactions within both centralized and decentralized control architectures. Moreover, the dynamic structure extension of DEVS enables runtime modifications of model topology, a critical capability for adaptive behavior in evolving IoCT environments.

Concerning continuity, DEVS provides a robust foundation for defining reusable building blocks and architectural patterns tailored to IoCT system design. Furthermore, DEVS execution engines exhibit versatility through implementation across a broad spectrum of technologies, including virtualization platforms, hardware accelerators, embedded systems, and even bio-inspired computational substrates.

5. Discussion

The following section summarizes the properties of the DEVS formalism, as illustrated in the preceding examples, to support its validity as a design language for IoCT systems—particularly in terms of model expressiveness and development continuity. This is followed by a review of other formalisms that can address these features with a comparison of their relative adequacy.

5.1. DEVS Properties for Model Expressiveness and Model Continuity

DEVS is a strong candidate for the expressiveness of IoCT system design due to its ability to model heterogeneous, dynamic, and event-driven environments while ensuring modularity, hierarchy, and formal validation. IoCT systems are inherently complex and distributed, involving diverse components that operate at different time scales and require flexible interaction mechanisms. DEVS offers key advantages that align well with these characteristics:

- **Modeling of Asynchronous and Event-Driven Behavior:** IoCT systems rely on asynchronous interactions among devices, sensors, and actuators. DEVS, as a discrete-event formalism, naturally represents systems where state changes occur at discrete time instants, making it well suited for capturing real-world IoCT dynamic [32]. It represents state changes at discrete time intervals, making it adept at modeling real-world dynamics in IoCT environments [33];
- **Hierarchical and Modular Structure:** DEVS enables hierarchical composition of models, allowing IoCT architectures to be designed in layers—such as edge, Fog, and

cloud computing—while maintaining encapsulation and interoperability between components [34]. This modularity enhances the reusability of models across different applications [32].

- **Separation of Concerns (Structure vs. Execution):** IoCT systems often require separating functional behavior from execution strategies. DEVS achieves this through its atomic models (defining component behavior) and coupled models (specifying interactions and execution flow), providing a clear separation between computation and communication [35].
- **Support for Concurrency and Synchronization:** IoCT components often involve multiple interacting subsystems that require concurrent processing. DEVS inherently supports parallel discrete-event simulation (P-DEVS), making it suitable for modeling concurrency, synchronization mechanisms, and conflict resolution in distributed IoCT systems [36].
- **Adaptability to Dynamic Environments:** IoCT applications demand adaptability due to changing conditions and evolving requirements. Dynamic structure DEVS (DS-DEVS) extends DEVS by allowing on-the-fly reconfiguration, which is essential for modeling adaptive behavior in IoCT networks [37]. Adaptive decision-making frameworks, such as the one proposed by Wang et al., utilize layers that sense, decide, and execute actions based on dynamic conditions [38].
- **Validation through Discrete-Event Simulation:** A critical aspect of IoCT system design is verifying whether execution strategies remain conformant with the intended functional model while incorporating real-world constraints. DEVS provides a rigorous simulation-based validation framework, allowing designers to test control strategies, real-time constraints, and system reliability before deployment [37].
- **Interoperability with Other Modeling Approaches:** IoCT system design often integrates multiple modeling paradigms, such as synchronous automata, Petri nets, and state machines [39]. DEVS can coexist with and complement these models, making it a flexible bridge for heterogeneous system design.

By capturing both system structure and execution dynamics while enabling modularity, adaptability, and validation, DEVS emerges as a powerful and expressive framework for IoCT system modeling and simulation.

Table 2 lists properties of the DEVS formalism related to model expressiveness, defined as its ability to express functional aspects of IoCT systems.

Table 2. Model expressiveness: ability to express required functionality in IoCT systems.

DEVS Properties	Expressiveness Features
DEVS atomic model functions	capture microscopic behaviors (messages, timing, decisions), express temporal interaction with sensors and actuators
DEVS hierarchical modular construction	supports incrementally verifiable functionality, expresses collaborative interaction in centralized and decentralized control, expresses the IoCT architecture, which is layered with sensor, Fog, and cloud layers
DEVS modularity	provides flexible support for the variable functionality required for AI/ML model analysis and retraining
DEVS temporal properties	express concurrent multiple streams of temporal events enabling simulation-based validation of the coordination and synchronization mechanisms
DEVS dynamic structure	enables structural changes needed for adaptive behavior
DEVS system-theory basis	supports definition of building blocks and architectural patterns for intelligent hybrid Cyber-Physical System design

Model continuity refers to the ability to transition a system description seamlessly across different stages of development—design, simulation, and execution—with minimal or no modifications. This is particularly important in IoCT system development, where models need to remain consistent across heterogeneous platforms and real-world constraints. DEVS provides a formal and modular approach that supports model continuity in the following ways:

- **Transition from Design to Simulation:** DEVS provides a formal specification that allows IoCT models to be directly simulated without reinterpreting their structure or behavior. The same model used in design can be executed in a discrete-event simulation environment, ensuring that functional behaviors (e.g., message passing, event synchronization, and timing constraints) are validated early. The hierarchical and modular nature of DEVS allows developers to incrementally refine their models while preserving core behavioral properties. For example, IoCT system architects can design DEVS models representing sensor interactions, data aggregation, and processing logic and then test these models in a simulation engine before deployment.
- **Transition from Simulation to Execution:** DEVS enables migration from simulated environments to real-world execution by transitioning from abstract simulation time to real-time execution. DEVS models can be mapped to real-time platforms, ensuring that the timing, coordination, and decision-making behaviors observed in simulation are maintained during execution. Through real-time DEVS (RT-DEVS), the same IoCT models can be integrated into embedded systems, middleware, and cloud environments without major alterations. For example, a DEVS-based traffic monitoring system tested in a simulation environment can be directly deployed onto real-world IoCT infrastructure while maintaining its event-driven behavior [40].
- **Support for Diverse Implementation Platforms:** DEVS models can be executed across a wide range of hardware and software platforms, including (i) embedded systems (IoCT devices and microcontrollers), (ii) edge and Fog computing environments, (iii) cloud-based IoCT platforms, and (iv) distributed simulation frameworks [41]. This adaptability ensures that the same IoCT model can be scaled and reused across multiple deployment scenarios. For example, a DEVS-based smart grid model can be tested in a cloud-based simulation environment and later deployed onto real-time distributed IoCT systems while preserving model fidelity.
- **Model Validation and Conflict Resolution:** DEVS simulation helps verify and validate execution strategies, ensuring that an IoCT system's operational behavior remains consistent with its design [42]. At the Operational Model level, DEVS supports conflict actuation management, helping resolve issues such as resource contention, sensor conflicts, and dynamic adaptation. For example, a smart building IoCT system modeled in DEVS can simulate conflicting temperature control settings before deployment, ensuring smooth operation.
- **Dynamic Adaptation and Evolution:** IoCT systems can reconfigure themselves autonomously, reducing the need for human intervention. This self-management is vital in complex environments, as highlighted in studies on self-adaptive software systems [43]. Through DS-DEVS, models can adapt to environmental changes in real time, allowing IoCT systems to be self-reconfigurable. This ensures that model continuity extends beyond initial deployment, supporting evolution and updates without requiring full redesigns. For example, an IoCT-based disaster response system modeled with DS-DEVS can dynamically adjust communication patterns and resource allocation in response to changing emergency conditions.

Table 3 lists transitions from the Design stage to the Simulation stage and from the latter to the Execution stage for which DEVS model continuity supports development of

IoCT systems. The table also considers the diversity of implementation that DEVS can work within.

Table 3. Model continuity: ability to transition the “Same” description from stage to stage.

Inter-Stage Transitions	DEVS Model Continuity Features
Migration from design to simulation	The DEVS simulation engine is coded in a variety of programming and higher-level languages for design and simulation.
Migration from simulation to execution	DEVS simulation engine can be transformed from its abstract time base to real-time bases, and DEVS models can be converted to hardware or middleware forms for real-time execution.
Diversity of implementation media	The DEVS simulation engine can be mapped to middleware implementations enabling straightforward integration with IoCT infrastructures. DEVS supports deployment across diverse hardware platforms with varying timing characteristics, underscoring its adaptability and portability. DEVS Execution engines can be implemented in diverse technologies such as virtualization, hardware, embedded systems, and bioware.

5.2. Comparison with Other Formalisms

It is important to compare other modeling formalisms that aim to achieve expressiveness and continuity in IoCT systems, including class diagrams, state machines, and Petri nets [44]. While each formalism has its strengths, they do not fully address the unique challenges of IoCT systems in the same way that DEVS does. This section examines how several popular formalisms compare with DEVS in terms of expressiveness, development continuity, and their ability to support key features of IoCT systems. Appendix A reviews these formalisms in relation to DEVS in detail. Table 4 compares these formalisms for expressiveness and model continuity in IoCT system design. In summary, while each of these formalisms offers valuable features—for static structure, control flow, concurrency, or complex systems modeling—they generally do not provide the same level of temporal expressiveness, modularity, and integrated execution support that DEVS delivers for dynamic IoCT systems.

Table 4. Comparison of different formalisms with DEVS.

Formalism	Strengths	Limitations Compared to DEVS
Class diagrams	Good for static structure modeling	Lacks temporal dynamics, reactivity, and execution models
State machines	Effective for sequential control and finite states	No support for concurrency, hierarchical design, or modularity
Petri nets	Excellent for concurrency and synchronization	Limited modularity, no inherent support for dynamic structures or execution strategies
SysML	Useful for complex system-of-systems modeling	Lacks native simulation support and real-time execution capabilities

Recent research has proposed alternative modeling architectures that aim to address some of the limitations of DEVS in terms of extensibility and dynamic interaction. For instance, Yuan et al. [45] introduced a three-layer component-based modeling method

that separates system models into components, entities, and systems. Their approach leverages behavior trees structured under the Postcondition–Precondition–Action (PPA) paradigm to modularize entity behaviors and employs a publish–subscribe mechanism to dynamically establish inter-entity communication. This architecture enables two-way control transfers and flexible model composition, which contrasts with the static port-based coupling in traditional DEVS models. While DEVS offers strong formal rigor and hierarchical modularity, the work of Yuan et al. highlights the potential of more dynamic and loosely coupled architectures to enhance model reusability and adaptability in evolving simulation scenarios. Including such perspectives strengthens the comparative analysis of modeling formalisms and supports the broader applicability of simulation-based development in IoCT systems.

A detailed comparison of DEVS with other modeling formalisms such as UML class diagrams, state machines, Petri nets, and SysML is provided in Appendix B.

5.3. Industrial Applications and Toolchain Integration

To illustrate DEVS's use in industrial-level toolchains, Xie et al. [46] introduced a modeling and simulation method for industrial product design using the X language, an integrated modeling language that combines system and physical models. Their approach leverages a DEVS simulator to derive simulations directly from X language models. Unlike traditional SysML-based workflows that require multiple software tools, this unified method simplifies modeling, digital simulation, and semi-physical simulation, thereby reducing complexity and improving efficiency. The effectiveness of the approach was demonstrated through a mechanical arm design case study. After modeling, a semi-physical simulation was conducted to verify the performance of a key component—the motor responsible for the base joint's rotation. The simulation results guided hardware selection, and subsequent testing confirmed that the motor met all performance requirements. This case highlights the potential of DEVS to support integrated, simulation-driven design processes in industrial contexts.

5.4. Validation in High-Performance Scientific Systems and Complex Industrial Applications

DEVS has also proven effective in addressing the challenges of large-scale, high-performance scientific systems. Bonaventura et al. [47] applied DEVS M&S to the ATLAS particle detector data acquisition system at CERN. This system faces extreme complexity, stringent timing constraints, and the need for interdisciplinary coordination and big-data processing. The authors reused and slightly adapted existing DEVS C++ simulation code to operate at the 100 kHz rate required by the high-level trigger event supervisor. The simulation was validated in a real-world deployment using nine event-processing units. The proposed scheduling algorithm, derived from simulation, outperformed conventional randomness-based and load-aware strategies, achieving two to four times lower latency while operating below 24 kHz. These results confirmed the predictive accuracy of the DEVS model and demonstrated its value in guiding the design and optimization of large-scale, real-time data systems.

The study by [48] proposes a real-time fault detection and diagnosis approach for CPS using DEVS. This work highlights the formalism's ability to model dynamic behaviors and integrate monitoring mechanisms in industrial distributed and safety-critical environments. The use of DEVS in this context illustrates its adaptability to complex industrial applications, thereby reinforcing its generality and robustness for IoCT system development.

6. Conclusions and Future Directions

This paper demonstrates that the DEVS formalism, when combined with the SES, provides the necessary expressiveness and development continuity to serve as a robust design language for IoCT systems. Illustrative examples—including home automation, solar sensor farm management, conflict resolution mechanisms, and dynamic unmanned swarm systems—demonstrate how DEVS supports adaptive and complex behaviors through hierarchical modularity, synchronization, and dynamic structural capabilities. Furthermore, the continuity inherent in DEVS enables seamless transitions from design and simulation to real-world implementation, making it a powerful tool for bridging the gap between conceptual models and operational systems.

Although the capability of DEVS to support simulation-based design of IoCT systems has been demonstrated, certain limitations in the design of such systems persist. In this context, model-driven engineering (MDE) [49] can play a significant role in augmenting DEVS-based design of such systems from the aspects of higher level support for expressiveness and continuity. More specifically, MDE provides a formal methodology to apply DEVS-based design capabilities to address the challenges of expressiveness and continuity in IoCT system design. On the expressiveness side, MDE supports the use of Domain-Specific Languages (DSLs), which provide customized abstractions to empower designers to articulate complex IoCT behaviors at a higher level, aligning the system design more closely with its functional requirements. Moreover, MDE promotes modular design and validation through formal methods, features that can significantly augment simulation-based equivalents when tractable. MDE also supports automated model transformations—converting models from design to simulation and then from simulation to execution—ensuring that a consistent system description is maintained at every stage. This consistency is further reinforced by traceability mechanisms that link different levels of abstraction, guaranteeing that changes in one phase are accurately propagated to subsequent phases.

By allowing for high-level abstractions, modular design, and seamless transitions between stages of development, MDE can augment the expressive and continuity capabilities of DEVS to significantly enhance the efficiency and accuracy of IoCT system development, from initial design through to real-world deployment.

On the other hand, more research needs to be carried out to improve the expressive capability of DEVS while improving the user experience at the same time. Developments such as non-standard DEVS point the way to such tool development [50].

Looking forward, the application of DEVS is expected to expand across increasingly diverse IoCT domains, highlighting its scalability, flexibility, and potential for integration with emerging technologies. By leveraging these strengths with the support of MDE methodology, DEVS is poised to play a pivotal role in the evolution of intelligent, adaptive systems within the rapidly advancing IoCT ecosystem.

Author Contributions: Investigation, B.P.Z., J.-F.S. and L.C.; writing—original draft preparation, B.P.Z., J.-F.S. and L.C.; revision, B.P.Z. and L.C.; software, B.P.Z., J.-F.S. and L.C.; writing—review and editing, J.-F.S. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Acknowledgments: The authors thank the reviewers for their highly constructive comments. We also express great appreciation for guidance in IoT and software applications from Valdemar Vicente Graciano Neto and Jean-Yves Tigli.

Conflicts of Interest: B.P. Zeigler declares that he is the inventor of DEVS and works for a commercial entity that is marketing DEVS-Based software. The other authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

DEVS	Discrete-Event System Specification
CPS	Cyber-Physical System
IoT	Internet of Things
IoCT	Internet of Cyber-Things
SES	System Entity Structure
M&S	Modeling and Simulation
DSL	Domain-Specific Languages
FSM	Finite-State Machine
DS-DEVS	Dynamic Structure DEVS
MBSE	Model-Based Systems Engineering
ACM	Actuation Conflict Management
CAIDE	Cloud-Based Analysis and Integration for Data Efficiency
RT-DEVS	Real-Time DEVS
USS	Unmanned Swarm System
UAV	Unmanned Autonomous Vehicle
PES	Pruned Entity Structure
P-DEVS	Parallel Discrete-Event System Specification
GUI	Graphical User Interface
API	Application Programming Interface
NN	Neural Network
RTSync	Real-Time Synchronization Corporation

Appendix A. DEVS and SES Foundations

DEVS models describe systems using a structured mathematical approach. Components can be either basic (atomic) or composed of other components (coupled) [51]. Figure A1 illustrates the types of DEVS models. Atomic models define how individual components behave in terms of inputs, outputs, and internal state changes.

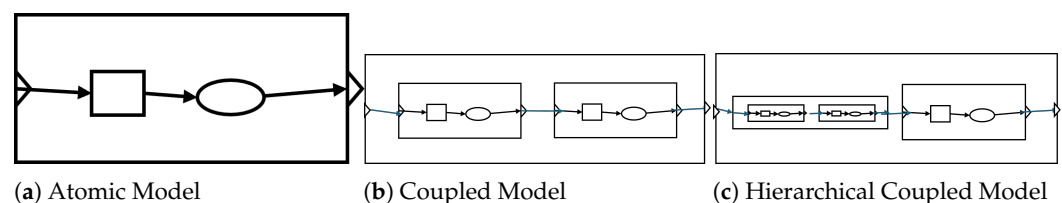


Figure A1. Illustration of DEVS models: (a) atomic model, (b) coupled model, and (c) hierarchical coupled model. Each level builds on the previous one to support scalable system design.

Coupled models define how components are connected—specifically, how outputs from one component become inputs to another. A key property of DEVS is *closure under coupling*, meaning that a coupled model behaves like an atomic model from the outside [3].

The System Entity Structure (SES) is a formal ontology for describing families of models [10]. It defines how systems can be decomposed into parts (aspects), how components can vary (specializations), and how they are connected. Figure A2 illustrates an SES for a smart IoCT application.

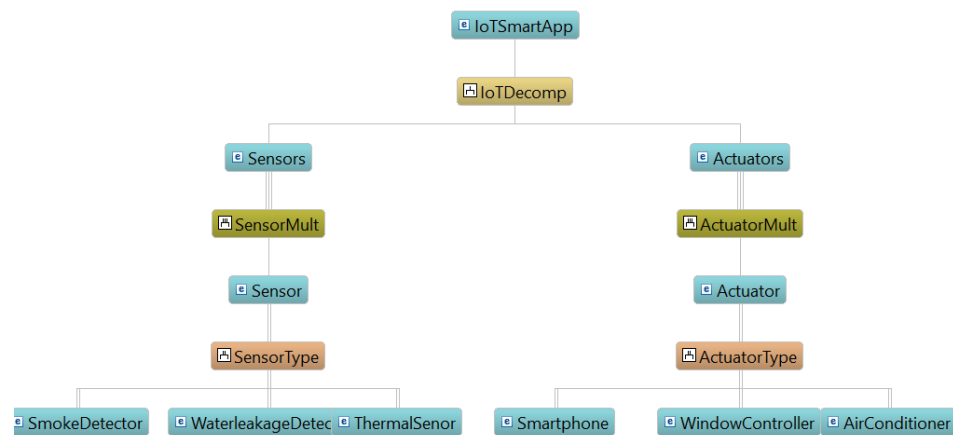


Figure A2. SES design of an IoT smart application. Sensors and actuators are defined using specialization.

From the IoTDecomp perspective, IoTSmartApp is made of Sensors and Actuators!
 From the SensorMult perspective, Sensors is made of more than one Sensor!
 From the ActuatorMult perspective, Actuators is made of more than one Actuator!
 Sensor can be SmokeDetector, WaterleakageDetection, or ThermalSensor in SensorType!
 Actuator can be Smartphone, WindowController, or AirConditioner in ActuatorType!

SES facilitates the generation of hierarchical DEVS models. Tools exist to write SES descriptions in constrained natural language and to prune them [10]. Pruned Entity Structures (PES) can be automatically converted into executable DEVS models [52]. This is essential for managing the growing complexity and variability of IoT systems [53].

Finally, DEVS universality allows modeling of discrete-event, continuous, and hybrid systems, including Event–Condition–Action (ECA) rules [54]. DS-DEVS extends this by supporting structural changes during simulation [27,55], enabling dynamic adaptation in evolving environments.

Appendix B. Comparing Modeling Formalisms and DEVS

There are several modeling formalisms—such as class diagrams, state machines, Petri nets, and SysML—that aim to provide expressiveness and continuity in IoT systems. Although each of these formalisms has its own strengths, they often fall short in addressing the unique challenges of IoT systems compared to DEVS. Below is a comparison of these popular formalisms with DEVS in terms of expressiveness, continuity, and support for key IoT features:

Appendix B.1. Class Diagrams (UML)

Class diagrams are commonly used in Unified Modeling Language (UML) to represent the structure of systems through classes, attributes, operations, and relationships between classes. They are valuable for defining the static structure of IoT systems, particularly for object-oriented design and database schema representation [56]. UML is ideal for capturing the hierarchical relationships and data organization within an IoT system (e.g., sensor data models and device classes). Widely Adopted: It is a well-understood formalism, especially in enterprise and software system design. Regarding the Limitations Compared to DEVS: Lack of temporal dynamics: Class diagrams do not inherently model time-dependent behavior or event-driven interactions that are crucial in IoT systems. This makes them less suitable for modeling asynchronous events and temporal dependencies. Limited Reactivity: Class diagrams are static and do not easily model reactive behavior—the ability of a system to respond to external stimuli or events in real time. No Explicit Support for Execution Models: Class diagrams do not specify how an IoT system behaves over

time, which limits their support for execution strategies or model validation in dynamic, event-driven environments.

Appendix B.2. State Machines

State machines or finite-state machines (FSMs) are widely used to model discrete states and state transitions based on input events. They are effective for describing control flow and sequential behavior, which makes them applicable to certain types of IoCT systems (e.g., simple control systems and state-based devices) [57]. State machines are good for modeling sequential logic and finite state transitions, which are common in IoCT devices (e.g., a smart thermostat with states like “heating”, “cooling”, and “idle”). Simple and Intuitive: Easy to understand and implement, making them suitable for small systems or components with straightforward behaviors. Regarding the Limitations Compared to DEVS: While state machines can model transitions, they lack the modular design inherent to DEVS. Complex IoCT systems that involve multiple interacting components may become difficult to manage using only state machines. No Support for Concurrency: Traditional state machines are inherently sequential and do not handle concurrent events well, which is a core feature in IoCT systems where multiple components interact simultaneously. Lack of Hierarchical Abstraction: DEVS allows for hierarchical modeling, which enables nested behavior and system decomposition—this is particularly useful in IoCT systems that have multiple layers (e.g., sensor networks, cloud services, and edge devices). State machines generally do not support this level of abstraction.

Appendix B.3. Petri Nets

Petri nets are a graphical and mathematical formalism used to model concurrent, asynchronous, and distributed systems. They have been used in modeling communication protocols, process control, and IoCT systems [58]. Petri nets are strong in modeling parallelism, concurrency, and synchronization of events, which is crucial in IoCT systems where multiple devices and sensors may operate simultaneously. Well Suited for Event-Driven Systems: They handle event-driven behaviors well and can model complex resource-sharing and token-passing mechanisms, which are common in IoCT systems. Regarding the Limitations Compared to DEVS: Lack of Modularity: While Petri nets can model concurrency, they do not support the modular composition of IoCT systems in the same way DEVS does. They can be complex to manage when dealing with large systems with many interacting components. Limited Focus on Execution Models: Petri nets model state transitions and events, but they do not inherently support execution strategies, such as mapping a model to real-time platforms or handling issues like timing constraints or adaptive reconfiguration. Partial Support for Dynamic Structure: While Petri nets can model system dynamics, they do not inherently support dynamic structure changes or self-adaptation in the same way DEVS with DS-DEVS does.

Appendix B.4. Systems Modeling Language

Systems Modeling Language (SysML), an extension of UML, is used for modeling complex systems of systems, and it includes state diagrams, activity diagrams, and block definition diagrams. SysML is frequently used in engineering and embedded systems [59]. SysML is suited for representing multi-domain systems (e.g., electrical, mechanical, and software components), which is useful in large IoCT systems. State Transitions and Behavior Modeling: Like UML state machines, SysML can represent state-based behaviors. Regarding the limitations compared to DEVS, SysML does not natively include simulation capabilities as part of the formalism. For IoCT systems, DEVS provides simulation and validation tools that allow for dynamic, event-driven analysis. Lack of Real-Time Behavior

Modeling: SysML does not inherently support the real-time execution of systems as DEVS does. IoCT systems often require not just simulation but also direct mapping to real-time execution environments, which DEVS provides seamlessly.

References

- Fortino, G.; Savaglio, C.; Spezzano, G.; Zhou, M. Internet of Things as System of Systems: A Review of Methodologies, Frameworks, Platforms, and Tools. *IEEE Trans. Syst. Man Cybern. Syst.* **2021**, *51*, 223–236. [\[CrossRef\]](#)
- Javed, A.; Malhi, A.; Kinnunen, T.; Främling, K. Scalable IoT Platform for Heterogeneous Devices in Smart Environments. *IEEE Access* **2020**, *8*, 211973–211985. [\[CrossRef\]](#)
- Zeigler, B.P.; Muzy, A.; Kofman, E. *Theory of Modeling and Simulation: Discrete Event & Iterative System Computational Foundations*; Academic Press: San Diego, CA, USA, 2018.
- Zeigler, B.P.; Mittal, S.T.M. MBSE with/out Simulation: State of the Art and Way Forward. *Systems* **2018**, *6*, 40. [\[CrossRef\]](#)
- Zeigler, B.; Mittal, S.; Traoré, M. Fundamental Requirements and DEVS Approach for Modeling and Simulation of Complex Adaptive System of Systems: Healthcare Reform. In Proceedings of the Symposium on Modeling and Simulation of Complexity in Intelligent, Adaptive and Autonomous Systems, Baltimore, MD, USA, 15–18 April 2021.
- Booker, L.; Forrest, S.; Mitchell, M.; Riolo, R. *Perspectives on Adaptation in Natural and Artificial Systems*; Oxford University Press: Oxford, UK, 2005. [\[CrossRef\]](#)
- Fattahi, A. IoT System Design Process and Main Components. In *IoT Product Design and Development: Best Practices for Industrial, Consumer, and Business Applications*; Wiley Science Publishers: New York, USA, 2023; pp. 95–161. [\[CrossRef\]](#)
- Alavi Fazel, I.; Wainer, G. Discrete Event System Specification for IoT Applications. *Sensors* **2024**, *24*, 7784. [\[CrossRef\]](#)
- Hu, X.; Zeigler, B.; Couretas, J. DEVS-on-a-chip: Implementing DEVS in real-time Java on a tiny Internet interface for scalable factory automation. In Proceedings of the 2001 IEEE International Conference on Systems, Man and Cybernetics, e-Systems and e-Man for Cybernetics in Cyberspace (Cat.No.01CH37236), Tucson, AZ, USA, 7–10 October 2001; Volume 5, pp. 3051–3056. [\[CrossRef\]](#)
- Zeigler, B.P.; Sarjoughian, H. *Guide to Modeling and Simulation of System of Systems*; Springer: New York, NY, USA, 2017.
- Anik, S.M.H.; Gao, X.; Zhong, H.; Wang, X.; Meng, N. Programming of Automation Configuration in Smart Home Systems: Challenges and Opportunities. *ACM Trans. Softw. Eng. Methodol.* **2025**. [\[CrossRef\]](#)
- Sehili, S.; Capocchi, L.; Santucci, J.F.; Lavirotte, S.; Tigli, J.Y. Discrete Event Modeling and Simulation for IoT Efficient Design Combining WComp and DEVSimPy Framework. In Proceedings of the 5th International Conference on Simulation and Modeling Methodologies, Technologies and Applications, Colmar, France, 21–23 July 2015; SIMULTECH 2015; pp. 26–34. [\[CrossRef\]](#)
- Risco-Martín, J.L.; Prado-Rujas, I.I.; Campoy, J.; Pérez, M.S.; Olcoz, K. Advanced simulation-based predictive modelling for solar irradiance sensor farms. *J. Simul.* **2024**, *19*, 265–282. [\[CrossRef\]](#)
- Zhang, L.; Zhao, C. *Modeling and Simulation Based Systems Engineering*; World Scientific: Singapore, 2023. Available online: <https://www.worldscientific.com/doi/pdf/10.1142/12960> (accessed on 11 June 2025). [\[CrossRef\]](#)
- Sarada, W.; Prashanth, K.; Mary, A.; Premkumar, K.; Vijayan, V.P.; Radha, G. IoT in Smart Homes: Enhancing Lifestyle Through Intelligent Automation and Control. In Proceedings of the 2025 International Conference on Automation and Computation (AUTOCOM), Dehradun, India, 4–6 March 2025; pp. 336–341. [\[CrossRef\]](#)
- Ansari, A.M.; Nazir, M.; Mustafa, K. Ontology-Based Classification and Detection of the Smart Home Automation Rules Conflicts. *IEEE Access* **2024**, *12*, 85072–85088. [\[CrossRef\]](#)
- Liu, R.; Wang, Z.; Garcia, L.; Srivastava, M. RemedioT: Remedial Actions for Internet-of-Things Conflicts. In Proceedings of the 6th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation (BuildSys '19), New York, NY, USA, 13–14 November 2019; Association for Computing Machinery: New York, NY, USA, 2019; pp. 101–110. [\[CrossRef\]](#)
- Capocchi, L.; Santucci, J.F.; Tigli, J.Y.; Gomin, T.; Lavirotte, S.; Rocher, G. Actuation Conflict Management in Internet of Things Systems DevOps: A Discrete Event Modeling and Simulation Approach. In *Proceedings of the Internet of Things*; Rey, G., Tigli, J.Y., Franquet, E., Eds.; Springer: Cham, Switzerland, 2025; pp. 189–206.
- Capocchi, L. DEVSimPy. Software Available on GitHub. 2024. Available online: <https://github.com/capocchi/DEVSimPy> (accessed on 11 June 2025).
- Capocchi, L.; Santucci, J.; Poggi, B.; Nicolai, C. DEVSimPy: A Collaborative Python Software for Modeling and Simulation of DEVS Systems. In Proceedings of the 2011 IEEE 20th International Workshops on Enabling Technologies: Infrastructure for Collaborative Enterprises, Paris, France, 27–29 June 2011; pp. 170–175. [\[CrossRef\]](#)
- Yentl Van Tendeloo and Hans Vangheluwe. An evaluation of DEVS simulation tools. *Simulation* **2017**, *93*, 103–121. [\[CrossRef\]](#)
- M, C.J.E.; Orna, G.; Daniel, K.; Doron, P.; Helmut, V. *Model Checking*; MIT Press: Cambridge, MA, USA, 2018.
- Fang, Z.; Fu, H.; Gu, T.; Qian, Z.; Jaeger, T.; Hu, P.; Mohapatra, P. A model checking-based security analysis framework for IoT systems. *High-Confid. Comput.* **2021**, *1*, 100004. [\[CrossRef\]](#)

24. Widyawati, D.K.; Ambarwari, A.; Wahyudi, A. Design and Prototype Development of Internet of Things for Greenhouse Monitoring System. In Proceedings of the 2020 3rd International Seminar on Research of Information Technology and Intelligent Systems (ISRITI), Yogyakarta, Indonesia, 10–11 December 2020; pp. 389–393. [\[CrossRef\]](#)
25. Harrand, N.; Fleurey, F.; Morin, B.; Husa, K.E. ThingML: A Language and Code Generation Framework for Heterogeneous Targets. In Proceedings of the ACM/IEEE 19th International Conference on Model Driven Engineering Languages and Systems, New York, NY, USA, 2–7 October 2016; MODELS '16, pp. 125–135. [\[CrossRef\]](#)
26. Dominici, A.; Capocchi, L.; De Gentili, E.; Santucci, J.F. Discrete Event Modeling and Simulation of Smart Parking Conflict Management. In Proceedings of the 24th International Congress on Modelling and Simulation, Sydney, Australia, 5–10 December 2021; Modsim '21, pp. 246–252. [\[CrossRef\]](#)
27. Zhang, W.; Li, Q.; Xu, X.; Li, W. Modeling and Simulation of Unmanned Swarm System Based on Dynamic Structure DEVS. *J. Phys. Conf. Ser.* **2024**, *2755*, 1–18. [\[CrossRef\]](#)
28. Banerjee, A.; Choppella, V. Control Software Engineering Approaches for Cyber-Physical Systems: A Systematic Mapping Study. *ACM Trans. Cyber-Phys. Syst.* **2025**, *9*, 1–33. [\[CrossRef\]](#)
29. Kate, C. Internet of Things and Beyond: Cyber-Physical Systems. 2016. Available online: <https://katecarruthers.com/2016/04/07/internet-of-things-and-beyond-cyber-physical-systems/> (accessed on 1 June 2025).
30. Zeigler, B. DEVS-based building blocks and architectural patterns for intelligent hybrid cyberphysical system design. *Information* **2021**, *12*, 531. [\[CrossRef\]](#)
31. Castro, R.; Marcosig, E.P.; Giribet, J.I. Simulation model continuity for efficient development of embedded controllers in cyber-physical systems. In *Complexity Challenges in Cyber Physical Systems: Using Modeling and Simulation (M&S) to Support Intelligence, Adaptation and Autonomy*; Springer: Cham, Switzerland, 2019; pp. 81–93.
32. Fazel, I.A.; Wainer, G. A DEVS-Based Methodology for Simulation and Model-Driven Development of IoT. In *Proceedings of the Simulation Tools and Techniques*; Guisado-Lizar, J.L., Riscos-Núñez, A., Morón-Fernández, M.J., Wainer, G., Eds.; Springer: Cham, Switzerland, 2024; pp. 3–17.
33. Rainey, L.; Holland, O. (Eds.) *Emergent Behavior in System of Systems Engineering: Real-World Applications*, 1st ed.; CRC Press: Boca Raton, FL, USA, 2022. [\[CrossRef\]](#)
34. Risco-Martín, J.L.; Mittal, S.; Henares, K.; Cardenas, R.; Arroba, P. xDEVS: A toolkit for interoperable modeling and simulation of formal discrete event systems. *Softw. Pract. Exp.* **2023**, *53*, 748–789. [\[CrossRef\]](#)
35. Capocchi, L.; Santucci, J.F.; Fericean, J.; Zeigler, B.P. DEVS Model Design for Simulation Web App Deployment. In Proceedings of the 2022 Winter Simulation Conference (WSC), Singapore, 11–14 December 2022; pp. 2154–2165. [\[CrossRef\]](#)
36. Trabes, G.G. Efficient DEVS Simulations Design on Heterogeneous Platforms. Doctoral Dissertation, Universidad Nacional de San Luis, San Luis, Argentina, 2023. [\[CrossRef\]](#)
37. Lee, E.; Seo, Y.D.; Kim, Y.G. Self-Adaptive Framework with Master–Slave Architecture for Internet of Things. *IEEE Internet Things J.* **2022**, *9*, 16472–16493. [\[CrossRef\]](#)
38. Wang, Y.; Zheng, L.; He, J.; Cui, Z. Adaptive IoT Decision Making in Uncertain Environments. In Proceedings of the 2023 IEEE International Conference on Smart Internet of Things (SmartIoT), Los Alamitos, CA, USA, 25–27 August 2023; pp. 265–269. [\[CrossRef\]](#)
39. An, H.; Park, W.; Park, S.; Lee, E. Logical Space Composition of IoT for a Scalable and Adaptable Smart Environment. In Proceedings of the 2024 International Conference on Information Networking (ICOIN), Ho Chi Minh City, Vietnam, 17–19 January 2024; pp. 614–618. [\[CrossRef\]](#)
40. Earle, B.; Bjornson, K.; Ruiz-Martin, C.; Wainer, G. Development of A Real-Time Devs Kernel: RT-Cadmium. In Proceedings of the 2020 Spring Simulation Conference (SpringSim), Virtual Event, 18–21 May 2020; pp. 1–12. [\[CrossRef\]](#)
41. Risco-Martín, J.L.; Mittal, S.; Fabero, J.C.; Malagón, P.; Ayala, J.L. Real-time hardware/software co-design using devs-based transparent M&S framework. In Proceedings of the Summer Computer Simulation Conference, San Diego, CA, USA, 24–27 July 2016; SCSC '16.
42. Hwang, K.; Lee, M.; Han, S.; Yoon, J.; You, Y.; Kim, S.; Nah, Y. The devs integrated development environment for simulation-based battle experimentation. *J. Korea Soc. Simul.* **2013**, *22*, 39–47. [\[CrossRef\]](#)
43. Matusek, D. Towards Resilient Execution of Adaptation in Decentralized Self-Adaptive Software Systems. In Proceedings of the 2022 IEEE International Conference on Autonomic Computing and Self-Organizing Systems Companion (ACSOS-C), Virtual, 19–23 September 2022; pp. 74–75. [\[CrossRef\]](#)
44. Alhirabi, N.; Rana, O.; Perera, C. Security and Privacy Requirements for the Internet of Things: A Survey. *ACM Trans. Internet Things* **2021**, *2*, 1–37. [\[CrossRef\]](#)
45. Yuan, H.; Yao, Y.; Tang, W.; Zhu, F. A High Extensible Modeling Method Using Three-Layer Component-Based Architecture. In Proceedings of the 2024 Winter Simulation Conference (WSC), Orlando, FL, USA, 15–18 December 2024; pp. 1–12. [\[CrossRef\]](#)
46. Xie, K.; Zhang, L.; Chen, Z.; Gu, P. Semi-physical Modeling and Simulation for Industrial Product Design Based on X Language. In Proceedings of the Asia Simulation Conference, Kobe, Japan, 17–20 September 2024; pp. 30–41.

47. Bonaventura, M.; Foguelman, D.; Castro, R. Discrete Event Modeling and Simulation-Driven Engineering for the ATLAS Data Acquisition Network. *Comput. Sci. Eng.* **2016**, *18*, 70–83. [\[CrossRef\]](#)
48. Boi-Ukeme, J.; Ruiz-Martin, C.; Wainer, G. Real-Time Fault Detection and Diagnosis of CPS Faults in DEVS. In Proceedings of the 2020 IEEE 6th International Conference on Dependability in Sensor, Cloud and Big Data Systems and Application (DependSys), Nadi, Fiji, 14–16 December 2020; pp. 57–64. [\[CrossRef\]](#)
49. Doddapaneni, K.; Ever, E.; Gemikonakli, O.; Malavolta, I.; Mostarda, L.; Muccini, H. A model-driven engineering framework for architecting and analysing wireless sensor networks. In Proceedings of the IEEE 2012 Third International Workshop on Software Engineering for Sensor Network Applications (SESENA), Zurich, Switzerland, 2 June 2012; pp. 1–7.
50. Junglas, P.; Jammer, D.; Pawletta, T.; Pawletta, S. Using component-based discrete-event modeling with NSA-DEVS—An invitation. In Proceedings of the ASIM 2024 Tagungsband Langbeiträge, 27th Symposium Simulationstechnik; Neubiberg, Germany, 4–6 September 2024; ASIM: Vienna, Austria, 2024.
51. Wymore, A. *A Mathematical Theory of Systems Engineering: The Elements*; Krieger: Huntington, NY, USA, 1967.
52. Bulcão-Neto, R.; Teixeira, P.; Lebttag, B.; Graciano-Neto, V.; Macedo, A.; Zeigler, B. Simulation of iot-oriented fall detection systems architectures for in-home patients. *IEEE Lat. Am. Trans.* **2023**, *21*, 16–26. [\[CrossRef\]](#)
53. Folkerts, H. An Architecture for Model Behavior Generation for Multiple Simulators. Ph.D. Thesis, University of Applied Sciences Wismar, Wismar, Germany, 2024. Available online: https://dokumente.ub.tu-clausthal.de/receive/clausthal_mods_00002606 (accessed on 1 June 2025). [\[CrossRef\]](#)
54. Samuel, K.G.; Bouare, N.D.M.; Maïga, O.; Traoré, M.K. A DEVS-based pivotal modeling formalism and its verification and validation framework. *Simulation* **2020**, *96*, 969–992. [\[CrossRef\]](#)
55. Uhrmacher, A.M. Dynamic structures in modeling and simulation: A reflective approach. *ACM Trans. Model. Comput. Simul.* **2001**, *11*, 206–232. [\[CrossRef\]](#)
56. Reggio, G. A UML-based proposal for IoT system requirements specification. In Proceedings of the 10th International Workshop on Modelling in Software Engineering, New York, NY, USA, 27–28 May 2018; MiSE '18, pp. 9–16. [\[CrossRef\]](#)
57. Xiao, R.; Wu, Z.; Wang, D. A Finite-State-Machine model driven service composition architecture for internet of things rapid prototyping. *Future Gener. Comput. Syst.* **2019**, *99*, 473–488. [\[CrossRef\]](#)
58. da Silva Fonseca, J.P.; de Sousa, A.R.; de Souza Tavares, J.J.P.Z. Modeling and controlling IoT-based devices' behavior with high-level Petri nets. *Procedia Comput. Sci.* **2023**, *217*, 1462–1469. [\[CrossRef\]](#)
59. Escamilla-Ambrosio, P.J.; Robles-Ramírez, D.A.; Tryfonas, T.; Rodríguez-Mota, A.; Gallegos-García, G.; Salinas-Rosales, M. IoTsecM: A UML/SysML Extension for Internet of Things Security Modeling. *IEEE Access* **2021**, *9*, 154112–154135. [\[CrossRef\]](#)

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.